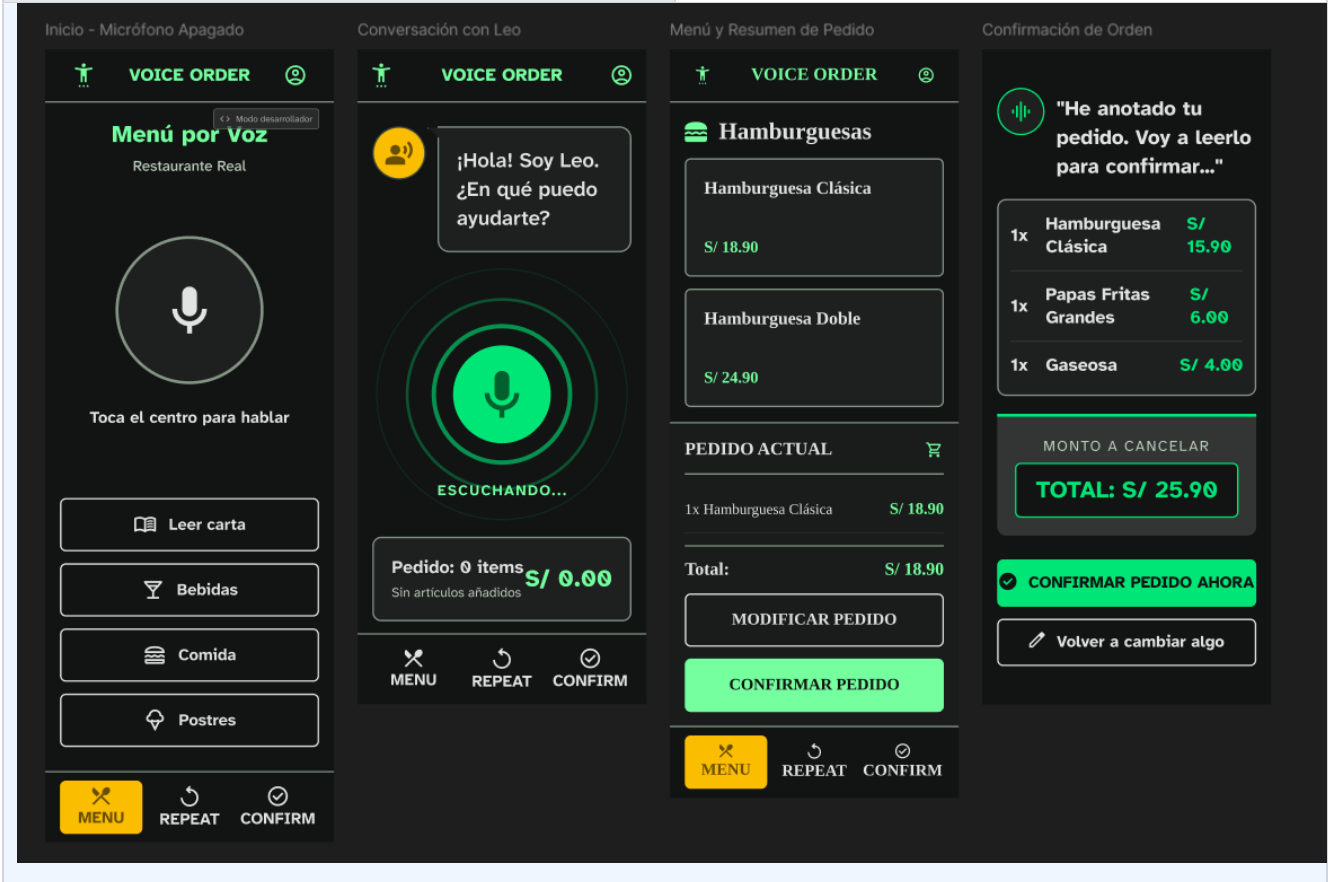


DOCUMENTACIÓN TÉCNICA DEL SISTEMA

Menú por Voz con IA para Restaurantes

Asistente conversacional accesible para consulta de menú y gestión de pedidos

Curso	Interacción Hombre Maquina
Docente	DAVID WILLIAM COTA SENCARA
Estudiante(s)	Flores Perez Paul Alexis
Institución	La Universidad Tecnológica del Perú
Periodo académico	9vo ciclo
Fecha	19/05/2026



Versión del documento: Avance 2

Índice del documento

- 1. Resumen ejecutivo
- 2. Problemática identificada
- 3. Objetivos del proyecto
- 4. Alcance del sistema
- 5. Propuesta de solución
- 6. Arquitectura del sistema
- 7. Modelo de base de datos
- 8. Módulos desarrollados
- 9. Endpoints principales
- 10. Prototipo e interfaz accesible
- 11. Evidencias de implementación
- 12. Pruebas realizadas
- 13. Limitaciones actuales
- 14. Próximos pasos
- 15. Conclusiones
- 16. Anexos sugeridos

1. Resumen ejecutivo

El presente documento describe el avance técnico del sistema “Menú por Voz con IA para Restaurantes”, una aplicación web orientada a facilitar la consulta del menú y la gestión de pedidos mediante interacción conversacional. El sistema está enfocado principalmente en personas con discapacidad visual o baja visión, por lo que la experiencia de usuario prioriza la comunicación auditiva, botones de gran tamaño, lectura de respuestas y un flujo guiado por voz.

A nivel de ingeniería de sistemas, el proyecto se plantea como una solución cliente-servidor compuesta por un frontend web, un backend API, una base de datos relacional en la nube y una capa conversacional que permite interpretar solicitudes básicas del usuario. El avance actual cuenta con backend funcional, base de datos en Supabase PostgreSQL, integración mediante Prisma, módulos de sesión, menú, pedido y chat, además de un frontend inicial accesible con asistente de voz llamado Leo.

2. Problemática identificada

En muchos restaurantes, el proceso de consulta del menú y toma de pedidos depende de menús físicos, cartas digitales visuales o de la atención directa del personal. Este modelo presenta barreras para usuarios con discapacidad visual, ya que dificulta la autonomía operativa para conocer productos, precios, variantes y opciones disponibles antes de realizar un pedido.

Desde el enfoque de ingeniería de sistemas, el problema no se limita a la lectura del menú, sino a la falta de un flujo digital accesible, estructurado y validado para registrar pedidos de forma segura. La ausencia de una interfaz conversacional adaptada puede generar demoras, dependencia del personal, errores de comunicación, falta de trazabilidad de interacciones y poca integración entre menú, pedido y confirmación.

3. Objetivos del proyecto

3.1 Objetivo general

Desarrollar un sistema web accesible basado en arquitectura cliente-servidor que permita consultar el menú de un restaurante y gestionar pedidos mediante interacción conversacional, integrando frontend, backend, base de datos relacional y funcionalidades de asistencia por voz.

3.2 Objetivos específicos

- Diseñar una base de datos relacional para administrar categorías, productos, variantes, modificadores, ingredientes, alérgenos, sesiones y pedidos.
- Implementar un backend en NestJS con TypeScript para centralizar las reglas de negocio del sistema.
- Integrar Supabase PostgreSQL como base de datos en la nube y Prisma como capa de acceso a datos.
- Implementar un flujo de pedido en estado borrador que permita agregar, modificar, eliminar y confirmar productos.
- Desarrollar un módulo de chat que interprete comandos básicos del usuario y los convierta en acciones sobre el pedido.
- Construir un frontend accesible orientado a interacción por voz, con retroalimentación auditiva y visual.
- Preparar una base técnica para futura integración con reconocimiento de voz, modelo de lenguaje local y síntesis de voz avanzada.

4. Alcance del sistema

4.1 Alcance implementado en el avance actual

- Creación de sesiones de atención.
- Listado de categorías y productos del menú.
- Consulta de productos con variantes, ingredientes y alérgenos.
- Creación y recuperación de pedidos en estado DRAFT.
- Agregar productos al pedido.
- Modificar cantidades.
- Eliminar productos.
- Confirmar pedidos.
- Bloqueo de modificaciones después de confirmar un pedido.
- Módulo de chat básico con intenciones simples.
- Frontend inicial con enfoque voice-first y asistente Leo.

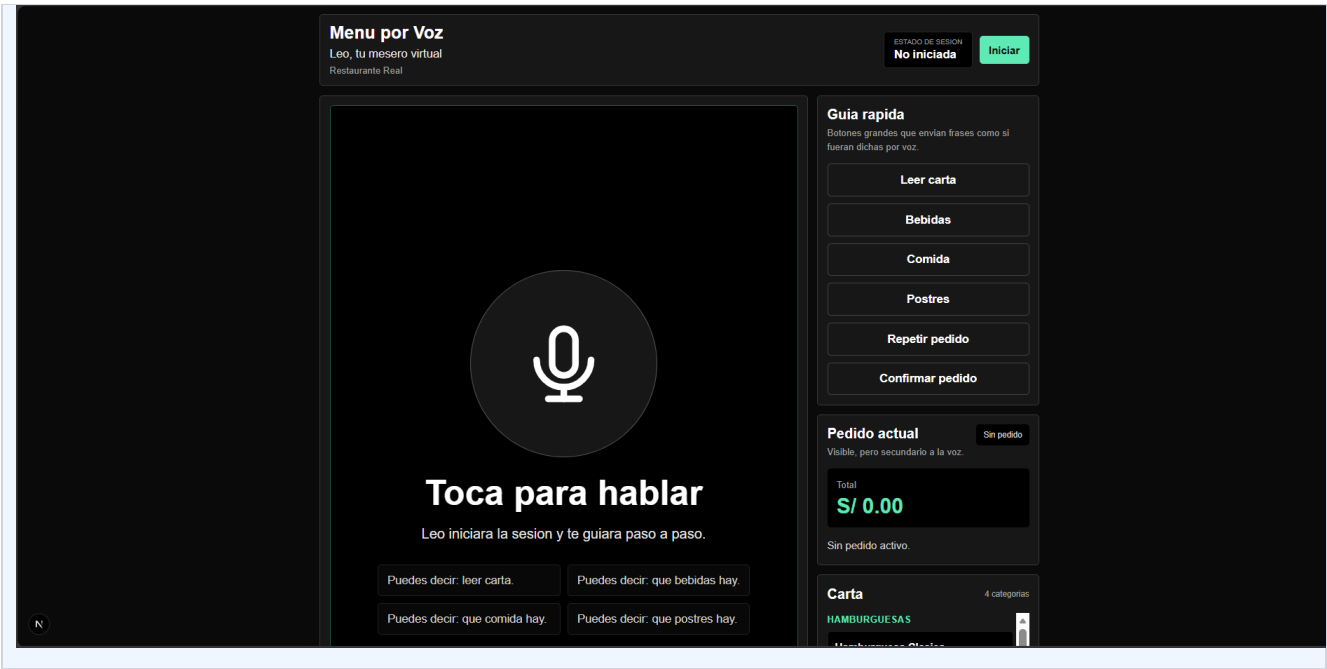
4.2 Fuera del alcance del avance actual

- Reconocimiento de voz definitivo mediante Whisper o motor local especializado.
- Integración completa con un modelo LLM local mediante Ollama.
- Pagos, facturación o emisión de comprobantes.
- Panel administrativo completo para el restaurante.
- Gestión de múltiples restaurantes.
- Control avanzado de stock por horario o inventario.

5. Propuesta de solución

La solución propuesta es una aplicación web accesible donde el usuario interactúa con Leo, un mesero virtual, para consultar la carta, conocer productos disponibles, agregar elementos al pedido, revisar el resumen y confirmar la orden. El sistema evita depender únicamente de elementos visuales, por lo que se prioriza un micrófono central de gran tamaño, mensajes hablados, estados claros y botones de acción accesibles.

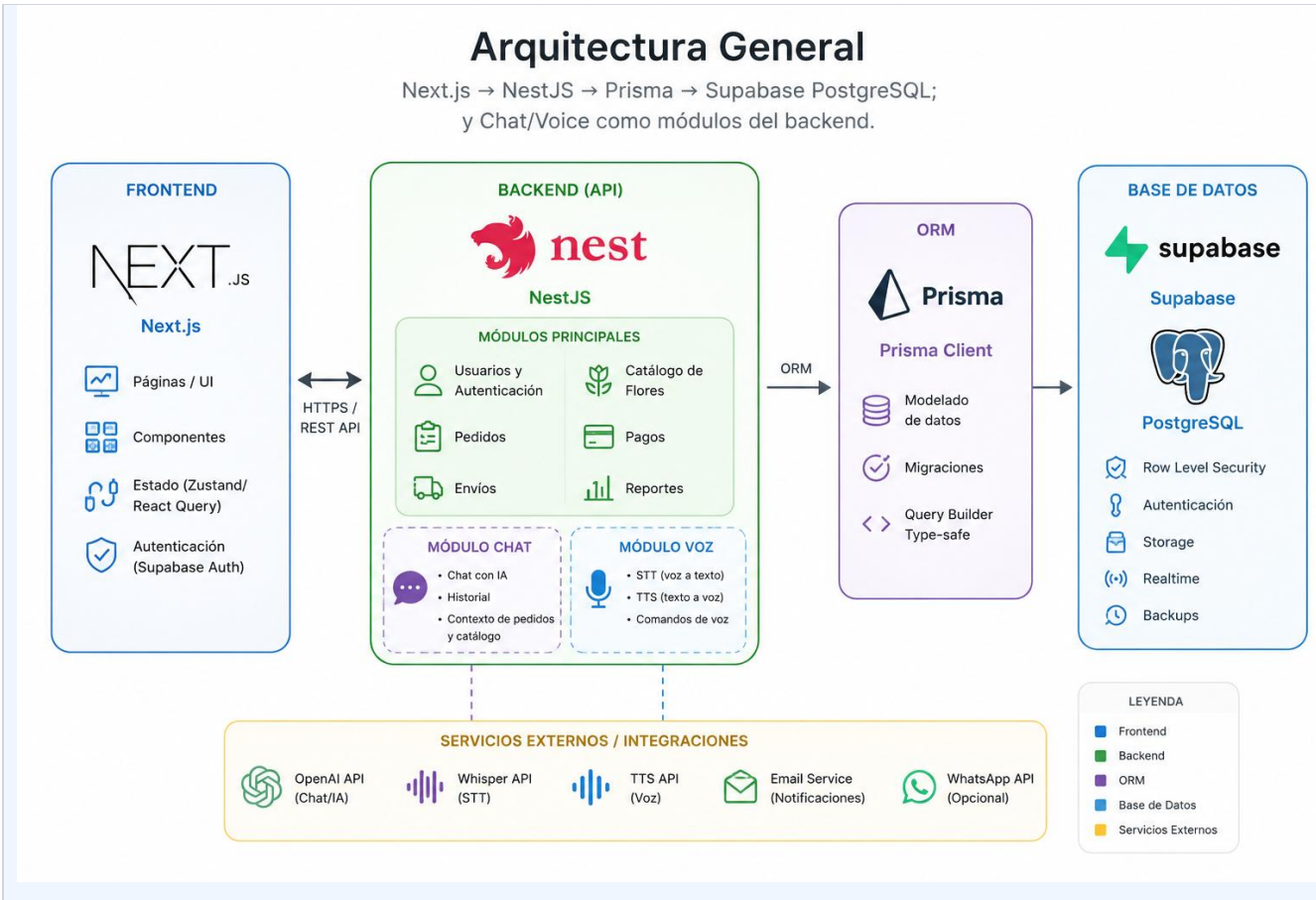
La lógica de negocio se concentra en el backend. El frontend no calcula precios ni totales; solo envía solicitudes al API. El backend consulta la base de datos, valida reglas del pedido, recalcula totales y registra eventos relevantes. Esta separación permite mayor control, mantenibilidad y trazabilidad.



6. Arquitectura del sistema

El sistema utiliza una arquitectura cliente-servidor con separación de responsabilidades entre interfaz, lógica de negocio y persistencia de datos.

Componente	Descripción técnica
Frontend - Next.js	Interfaz web accesible. Gestiona la interacción visual, mensajes del asistente y síntesis de voz mediante APIs del navegador.
Backend - NestJS	API central del sistema. Administra sesiones, menú, pedidos, chat y reglas de negocio.
Prisma	Capa de acceso a datos y cliente tipado para interactuar con PostgreSQL.
Supabase PostgreSQL	Base de datos relacional en la nube donde se almacenan menú, sesiones, pedidos e interacciones.
SpeechSynthesis	Funcionalidad del navegador usada para leer respuestas del asistente en voz alta.
Futura capa IA	Integración proyectada con Ollama o modelo local para interpretación más avanzada del lenguaje natural.



6.1 Flujo general de interacción

- El usuario inicia una sesión desde el frontend.
- El sistema crea o recupera una sesión activa en el backend.
- El usuario consulta el menú o solicita un pedido mediante texto/voz.
- El módulo chat interpreta la intención básica.
- El backend consulta el menú y/o modifica el pedido DRAFT.
- El backend responde con un mensaje del asistente.
- El frontend muestra y lee la respuesta mediante síntesis de voz.
- El usuario puede repetir, modificar o confirmar el pedido.

7. Modelo de base de datos

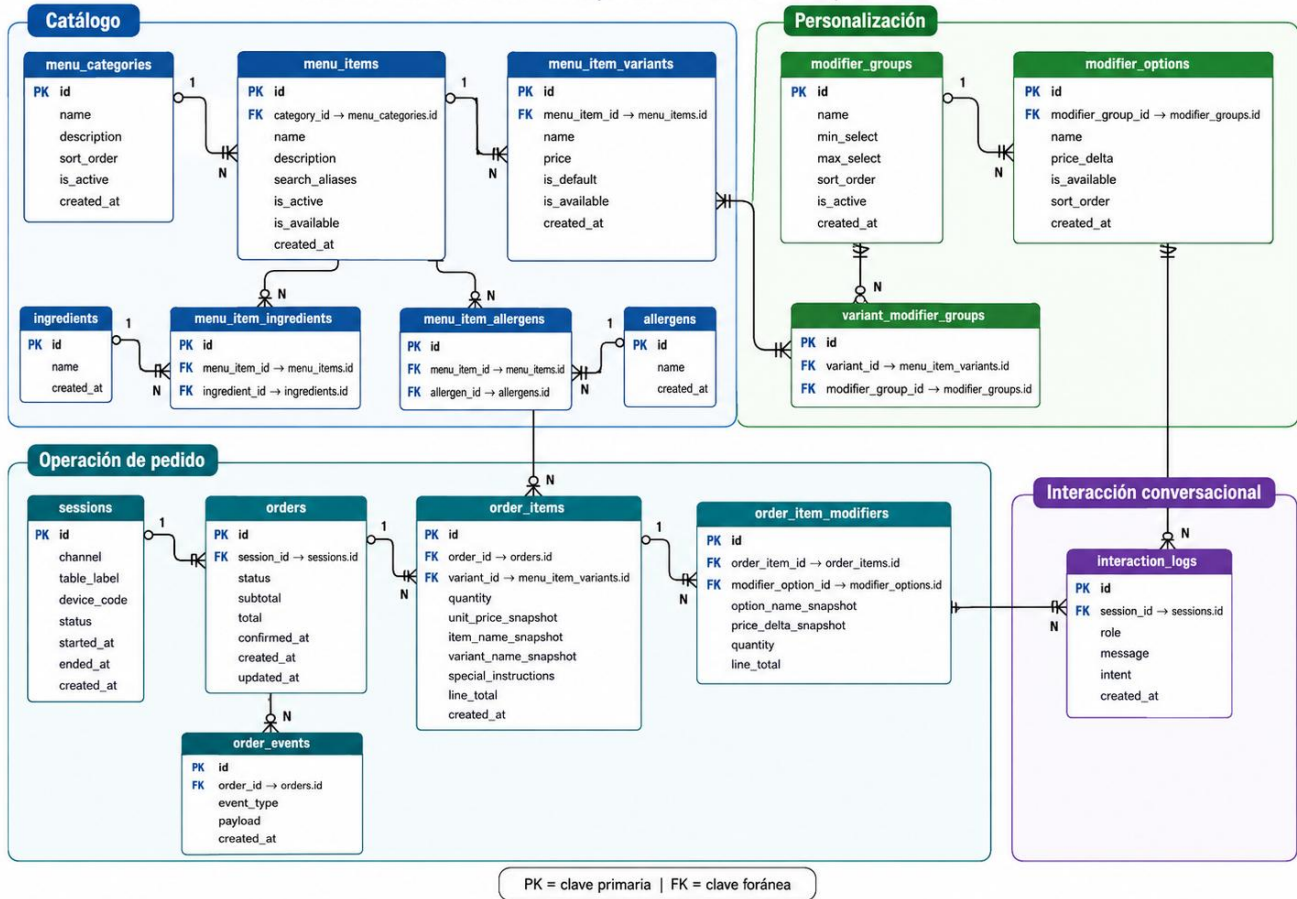
La base de datos fue diseñada para representar productos del menú, variantes, opciones, pedidos, sesiones e interacciones. La estructura permite registrar pedidos en estado DRAFT antes de confirmarlos, lo que habilita agregar, modificar o eliminar productos durante la conversación.

Tabla	Función	Uso dentro del sistema
sessions	Registra sesiones de atención.	Permite asociar conversación y pedido activo.
categories	Agrupar productos del menú.	Permite listar el menú por secciones.

menu_items	Representa productos base.	Almacena nombre, descripción, disponibilidad y alias de búsqueda.
menu_item_variants	Define presentaciones vendibles.	Permite manejar tamaños, combos o variantes con precio.
modifier_groups / modifier_options	Representa extras y opciones.	Base para seleccionar acompañamientos, extras o configuraciones.
ingredients / allergens	Describe composición y alérgenos.	Permite responder consultas sobre ingredientes y restricciones alimentarias.
orders	Cabecera del pedido.	Controla estado DRAFT, CONFIRMED y totales.
order_items	Líneas del pedido.	Registra producto, cantidad, snapshot y subtotal.
order_item_modifiers	Extras seleccionados por ítem.	Permite registrar modificadores elegidos.
interaction_logs	Historial conversacional.	Guarda mensajes de usuario y asistente.
order_events	Auditoría del pedido.	Registra cambios relevantes del pedido.

Diagrama Entidad-Relación - Menú por Voz con IA

Base de datos del sistema de pedidos conversacional para restaurante

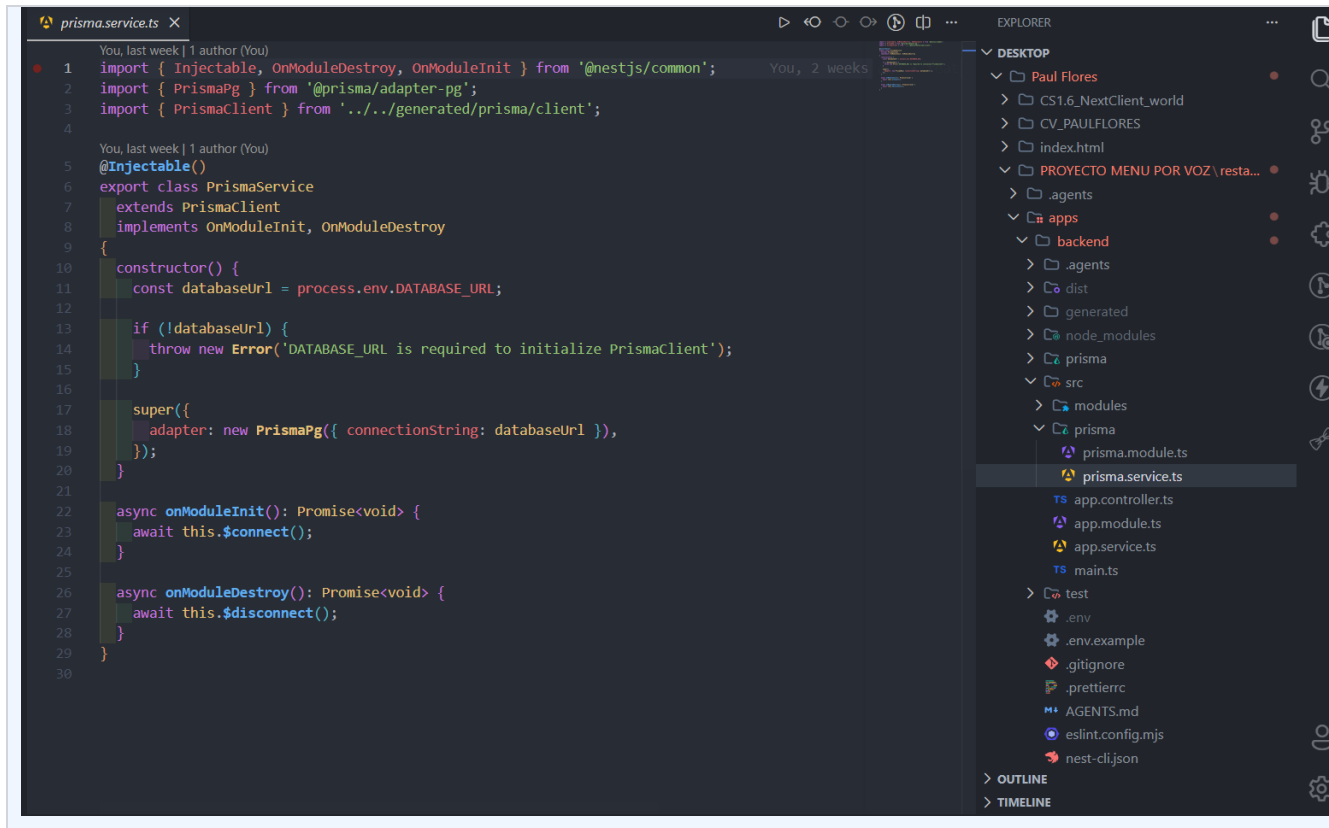


Módulo	Responsabilidad	Estado
--------	-----------------	--------

9. Evidencia de código relevante

En esta sección se recomienda insertar capturas o fragmentos de código representativos. No es necesario pegar todo el proyecto; se deben mostrar partes clave que evidencian arquitectura, conexión, validación y

Conexión Prisma



Módulo de sesión

The image displays two screenshots of a code editor, likely Visual Studio Code, showing TypeScript code for a session module. The top screenshot shows the `session.controller.ts` file, and the bottom screenshot shows the `session.service.ts` file. Both files are part of a project named "PROYECTO MENU POR VOZ".

session.controller.ts

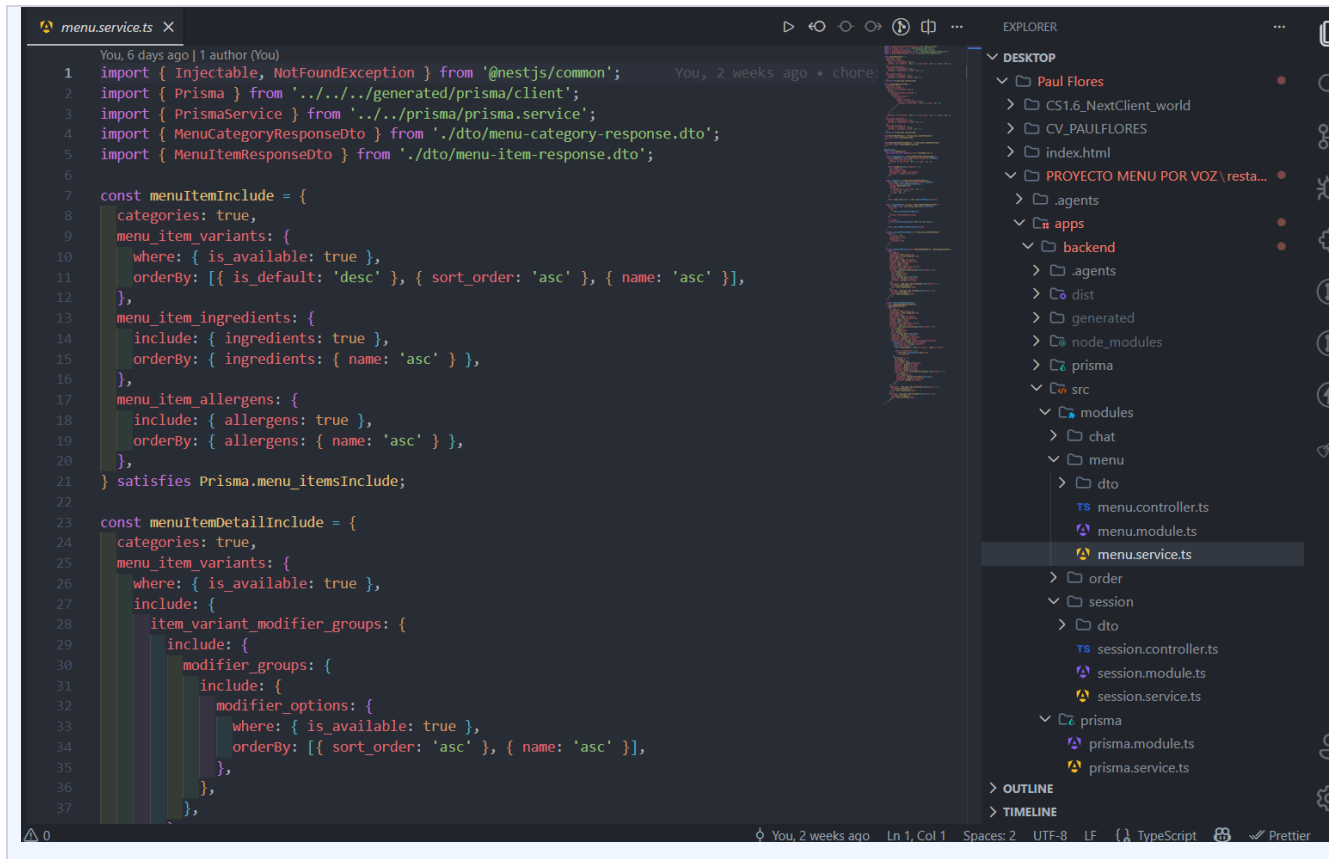
```
1 import { Body, Controller, Post } from '@nestjs/common';
2 import { CreateSessionDto } from '../dto/create-session.dto';
3 import { SessionResponseDto } from '../dto/session-response.dto';
4 import { SessionService } from './session.service';
5
6 @Controller('sessions')
7 export class SessionController {
8   constructor(private readonly sessionService: SessionService) {}
9
10  @Post()
11  create(@Body() dto: CreateSessionDto): Promise<SessionResponseDto> {
12    return this.sessionService.create(dto);
13  }
14 }
15
```

session.service.ts

```
1 import { Injectable } from '@nestjs/common';
2 import { session_status } from '../../generated/prisma/enums';
3 import { PrismaService } from '../../prisma/prisma.service';
4 import { CreateSessionDto } from '../dto/create-session.dto';
5 import { SessionResponseDto } from '../dto/session-response.dto';
6
7 @Injectable()
8 export class SessionService {
9   constructor(private readonly prisma: PrismaService) {}
10
11  async create(dto: CreateSessionDto): Promise<SessionResponseDto> {
12    const session = await this.prisma.sessions.create({
13      data: {
14        channel: dto.channel?.trim() || 'voice',
15        table_label: dto.tableLabel?.trim() || null,
16        device_code: dto.deviceCode?.trim() || null,
17        status: session_status.ACTIVE,
18      },
19    });
20
21    return {
22      id: session.id,
23      channel: session.channel,
24      tableLabel: session.table_label,
25      deviceCode: session.device_code,
26      status: session.status,
27      lastInteractionAt: session.last_interaction_at,
28      createdAt: session.created_at,
29      updatedAt: session.updated_at,
30    };
31  }
32 }
33
```

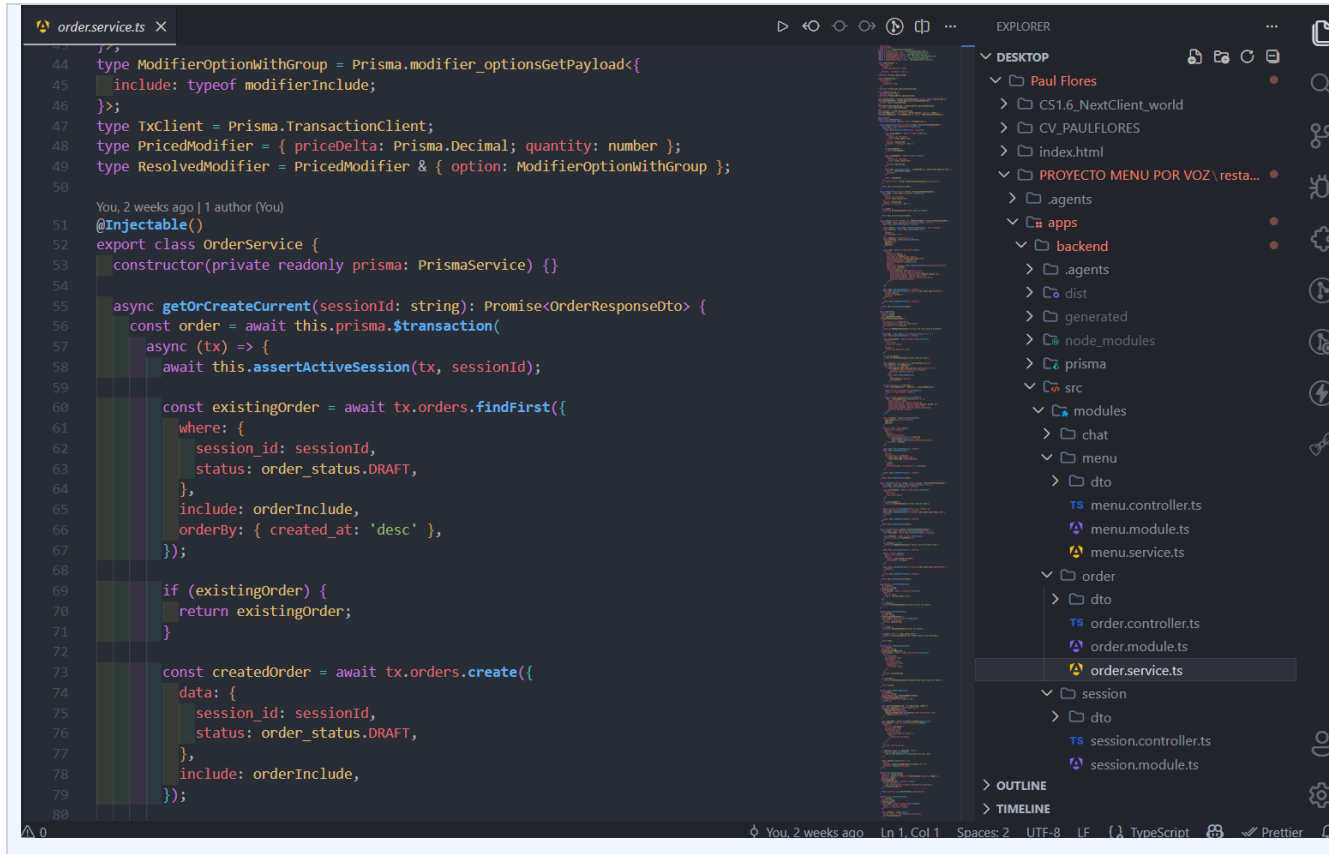
The Explorer panel on the right shows the project structure, including the `session` folder and the `session` module files.

Consulta de menú



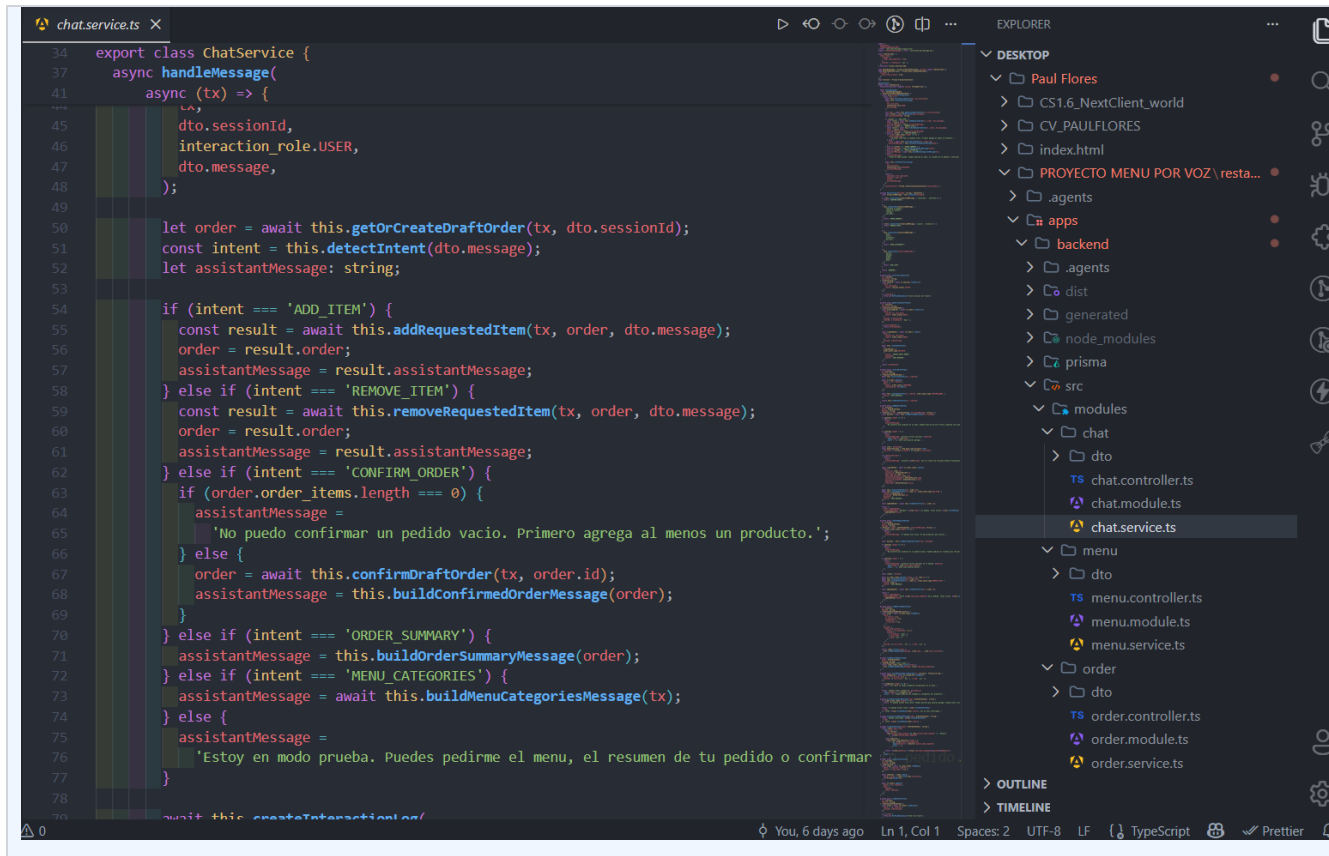
```
1 import { Injectable, NotFoundException } from '@nestjs/common';
2 import { Prisma } from '../../generated/prisma/client';
3 import { PrismaService } from '../../prisma/prisma.service';
4 import { MenuCategoryResponseDto } from '../dto/menu-category-response.dto';
5 import { MenuItemResponseDto } from '../dto/menu-item-response.dto';
6
7 const menuItemInclude = {
8   categories: true,
9   menu_item_variants: {
10     where: { is_available: true },
11     orderBy: [{ is_default: 'desc' }, { sort_order: 'asc' }, { name: 'asc' }],
12   },
13   menu_item_ingredients: {
14     include: { ingredients: true },
15     orderBy: { ingredients: { name: 'asc' } },
16   },
17   menu_item_allergens: {
18     include: { allergens: true },
19     orderBy: { allergens: { name: 'asc' } },
20   },
21 } satisfies Prisma.menu_itemsInclude;
22
23 const menuItemDetailInclude = {
24   categories: true,
25   menu_item_variants: {
26     where: { is_available: true },
27     include: {
28       item_variant_modifier_groups: {
29         include: {
30           modifier_groups: {
31             include: {
32               modifier_options: {
33                 where: { is_available: true },
34                 orderBy: [{ sort_order: 'asc' }, { name: 'asc' }],
35               },
36             },
37           },
38         },
39       },
40     },
41   },
42 }
```

Flujo de pedidos



```
44 type ModifierOptionWithGroup = Prisma.modifier_optionsGetPayload<{
45   include: typeof modifierInclude;
46 }>;
47 type TxClient = Prisma.TransactionClient;
48 type PricedModifier = { priceDelta: Prisma.Decimal; quantity: number };
49 type ResolvedModifier = PricedModifier & { option: ModifierOptionWithGroup };
50
51 You, 2 weeks ago | 1 author (You)
52 @Injectable()
53 export class OrderService {
54   constructor(private readonly prisma: PrismaService) {}
55
56   async getOrCreateCurrent(sessionId: string): Promise<OrderResponseDto> {
57     const order = await this.prisma.$transaction(
58       async (tx) => {
59         await this.assertActiveSession(tx, sessionId);
60
61         const existingOrder = await tx.orders.findFirst({
62           where: {
63             session_id: sessionId,
64             status: order_status.DRAFT,
65           },
66           include: orderInclude,
67           orderBy: { created_at: 'desc' },
68         });
69
70         if (existingOrder) {
71           return existingOrder;
72         }
73
74         const createdOrder = await tx.orders.create({
75           data: {
76             session_id: sessionId,
77             status: order_status.DRAFT,
78           },
79           include: orderInclude,
80         });
81       }
82     );
83     return order;
84   }
85
86   async assertActiveSession(tx: Prisma.TransactionClient, sessionId: string) {
87     const session = await tx.sessions.findFirst({
88       where: {
89         id: sessionId,
90       },
91     });
92     if (!session) {
93       throw new NotFoundException('Session not found');
94     }
95     if (session.status !== session_status.ACTIVE) {
96       throw new BadRequestException('Session is not active');
97     }
98   }
99 }
```

Chat básico



```
34 export class ChatService {
35   async handleMessage(
41     async (tx) => {
42       const {
45         dto.sessionId,
46         interaction_role.USER,
47         dto.message,
48       };
49     };
50     let order = await this.getOrCreateDraftOrder(tx, dto.sessionId);
51     const intent = this.detectIntent(dto.message);
52     let assistantMessage: string;
53
54     if (intent === 'ADD_ITEM') {
55       const result = await this.addRequestedItem(tx, order, dto.message);
56       order = result.order;
57       assistantMessage = result.assistantMessage;
58     } else if (intent === 'REMOVE_ITEM') {
59       const result = await this.removeRequestedItem(tx, order, dto.message);
60       order = result.order;
61       assistantMessage = result.assistantMessage;
62     } else if (intent === 'CONFIRM_ORDER') {
63       if (order.order_items.length === 0) {
64         assistantMessage =
65           'No puedo confirmar un pedido vacío. Primero agrega al menos un producto.';
66       } else {
67         order = await this.confirmDraftOrder(tx, order.id);
68         assistantMessage = this.buildConfirmedOrderMessage(order);
69       }
70     } else if (intent === 'ORDER_SUMMARY') {
71       assistantMessage = this.buildOrderSummaryMessage(order);
72     } else if (intent === 'MENU_CATEGORIES') {
73       assistantMessage = await this.buildMenuCategoriesMessage(tx);
74     } else {
75       assistantMessage =
76         'Estoy en modo prueba. Puedes pedirme el menú, el resumen de tu pedido o confirmar
77     }
78   }
79   await this.createInteractionLog(
```

0 You, 6 days ago Ln 1, Col 1 Spaces: 2 UTF-8 LF TypeScript Prettier

Frontend accesible

page.tsx

```
118 const apiUrl = (  
119   process.env.NEXT_PUBLIC_API_URL || "http://localhost:3001"  
120 ).replace(/\/$/, "");  
121  
122 const leoIntro =  
123   "Bienvenido al Restaurante Real. Soy Leo, tu mesero virtual. Te ayudare a consultar la cart  
124  
125 const micActivatedMessage = "Microfono activado. Te escucho.";   
126 const micOffMessage = "Microfono apagado. Toca para hablar nuevamente.";   
127 const noVoiceMessage =  
128   "No detecte tu voz. Toca nuevamente el centro de la pantalla para intentarlo.";   
129 const noPermissionMessage =  
130   "No tengo permiso para usar el microfono. Activa el permiso del navegador o usa el modo pru  
131 const notUnderstoodMessage =  
132   "No pude entenderte. Puedes decir: leer carta, bebidas, comida o repetir mi pedido.";   
133 const unsupportedRecognitionMessage =  
134   "Este navegador no permite reconocimiento de voz aqui. Usa Chrome o Edge, activa permisos,  
135  
136 const quickActions = [  
137   { label: "Leer carta", message: "leer carta" },  
138   { label: "Bebidas", message: "que bebidas hay" },  
139   { label: "Comida", message: "que comida hay" },  
140   { label: "Postres", message: "que postres hay" },  
141   { label: "Repetir pedido", message: "repiteme mi pedido" },  
142   { label: "Confirmar pedido", message: "confirmo mi pedido" },  
143 ];  
144  
145 const voiceStatus: Record<VoiceState, { title: string; helper: string }> = {  
146   start: {  
147     title: "Toca para hablar",  
148     helper: "Leo iniciara la sesion y te guiara paso a paso.",  
149   },  
150   off: {  
151     title: "Microfono apagado",  
152     helper: "Toca el centro de la pantalla para hablar con Leo.",  
153   },  
154   activating: {  
155     title: "Microfono activado",
```

EXPLORER

DESKTOP

Paul Flores

CS1.6_NextClient_world

CV_PAULFLORES

index.html

PROYECTO MENU POR VOZ \resta...

.agents

apps

backend

frontend

.next

app

favicon.ico

globals.css

layout.tsx

page.tsx

node_modules

.bin

@alloc

@babel

@emnapi

@eslint

@eslint-community

@humanfs

@humanwhocodes

@img

@jridgewell

@napi-rs

@next

@nodelib

@nolyfill

OUTLINE

TIMELINE

0

You, yesterday

Paul Alexis Flores Perez (1 day ago)

Ln 109, Col 53

Spaces: 2

UTF-8

LF

TypeScript JSX

Prettier

10. Endpoints principales del backend

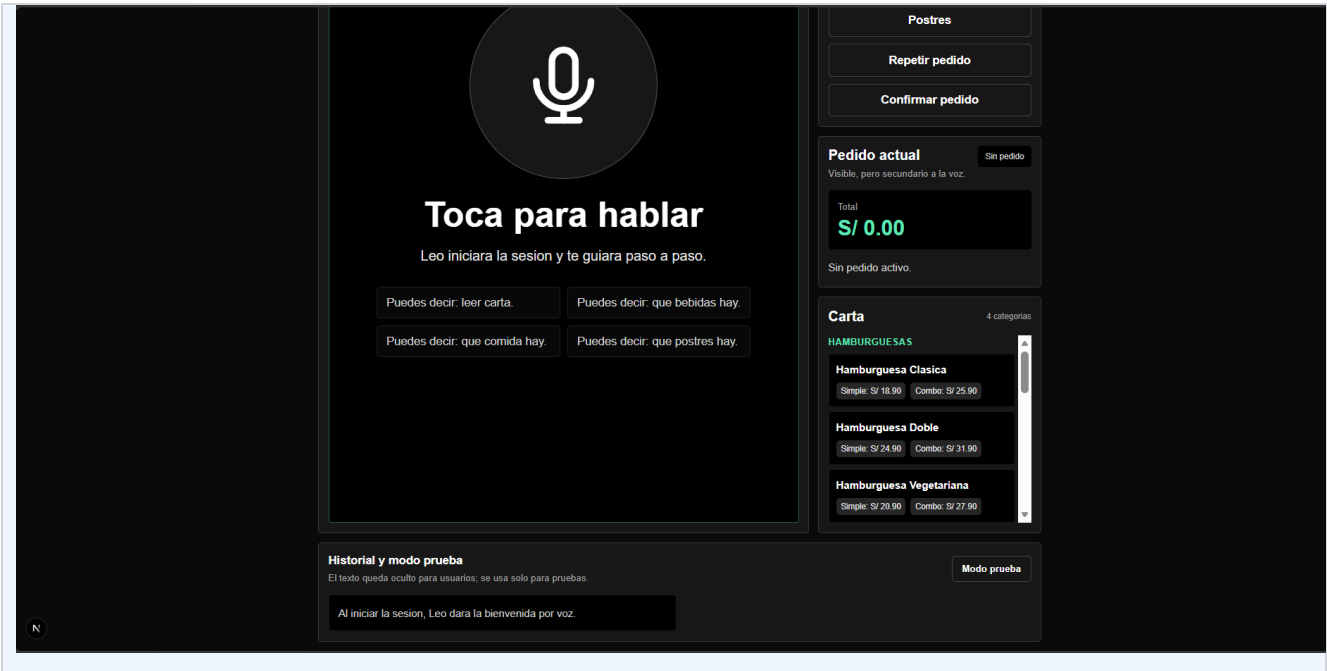
Método	Ruta	Propósito	Estado
POST	/sessions	Crear sesión activa de atención.	Probado
GET	/menu/categories	Listar categorías del menú.	Probado
GET	/menu/items	Listar productos disponibles.	Probado
GET	/menu/items/:id	Consultar detalle de producto.	Probado / mejorable
POST	/orders/current	Crear o recuperar pedido DRAFT.	Probado
GET	/orders/current/:sessionId	Obtener pedido actual de una sesión.	Probado
POST	/orders/:orderId/items	Agregar producto al pedido.	Probado
PATCH	/orders/:orderId/items/:itemId	Modificar item del pedido.	Probado
DELETE	/orders/:orderId/items/:itemId	Eliminar item del pedido.	Probado
POST	/orders/:orderId/confirm	Confirmar pedido.	Probado
POST	/chat/message	Enviar mensaje al asistente Leo.	Probado

Avance 2 - Proyecto académico

11. Prototipo e interfaz accesible

El frontend fue diseñado como un prototipo funcional centrado en voz. La interfaz no debe depender de que el usuario escriba texto, ya que el objetivo principal es atender a personas con discapacidad visual o baja visión. Por ello, el micrófono se plantea como el elemento principal de interacción y el asistente Leo guía al usuario con mensajes hablados.

- Micrófono central de gran tamaño.
- Estados visibles y auditivos del sistema.
- Lectura de respuestas mediante speechSynthesis.
- Botones rápidos: leer carta, bebidas, comida, postres, repetir pedido y confirmar.
- Modo prueba para ingresar texto, pero no como interacción principal.
- Panel de pedido actual como apoyo visual para presentación y pruebas.



12. Pruebas realizadas

Se realizaron pruebas manuales sobre los endpoints del backend y sobre el frontend de prueba. La validación se centró en el flujo base del sistema: crear sesión, consultar menú, generar pedido, modificarlo, confirmarlo y bloquear cambios posteriores.

Caso de prueba	Entrada	Resultado esperado	Estado
Crear sesión	POST /sessions	Sesión ACTIVE con UUID.	Correcto
Listar menú	GET /menu/items	Productos con variantes y precios.	Correcto
Crear pedido DRAFT	POST /orders/current	Pedido vacío en estado DRAFT.	Correcto

Agregar producto	POST /orders/:id/items	Item agregado y total recalculado.	Correcto
Modificar cantidad	PATCH /orders/:id/items/:itemId	Cantidad actualizada y total recalculado.	Correcto
Eliminar producto	DELETE /orders/:id/items/:itemId	Item eliminado y total actualizado.	Correcto
Confirmar pedido	POST /orders/:id/confirm	Pedido cambia a CONFIRMED.	Correcto
Modificar confirmado	PATCH posterior a confirmación	Error 409: solo pedidos DRAFT se modifican.	Correcto
Chat resumen	POST /chat/message: "repítame mi pedido"	Leo responde con resumen del pedido.	Correcto
Chat agregar producto	POST /chat/message: "quiero una hamburguesa"	Producto agregado por intención ADD_ITEM.	Correcto

13. Limitaciones actuales

- El reconocimiento de voz depende actualmente de la compatibilidad del navegador. En las pruebas, Microsoft Edge presentó mejor comportamiento que Brave.
- El chat interpreta comandos simples mediante reglas básicas; todavía no utiliza un modelo de lenguaje local.
- La selección de variantes, cantidades por lenguaje natural, extras y modificadores requiere mejoras adicionales.
- El sistema aún no cuenta con panel administrativo completo para edición del menú.
- El frontend es un prototipo funcional, no una versión final de producción.
- La voz del asistente depende de speechSynthesis y de las voces instaladas en el sistema operativo/navegador.

14. Próximos pasos

- Mejorar la interpretación conversacional para cantidades, variantes y modificadores.
- Integrar un modelo LLM local mediante Ollama para comprender mejor lenguaje natural.
- Implementar o consolidar reconocimiento de voz mediante una solución más estable.
- Incorporar lectura progresiva del menú por categorías para mejorar la accesibilidad.
- Desarrollar un panel administrativo para cargar o editar el menú real del restaurante.
- Validar el sistema con un menú real y usuarios de prueba.
- Preparar documentación de pruebas y evidencias para presentación final.

15. Conclusiones

El avance actual demuestra la viabilidad técnica del sistema propuesto. Se implementó una base funcional con backend, base de datos, flujo de pedidos, chat básico y frontend accesible. El sistema ya permite

consultar menú, crear sesiones, administrar pedidos en estado borrador, confirmar órdenes y utilizar un asistente conversacional básico.

Desde el punto de vista de ingeniería de sistemas, el proyecto presenta una arquitectura modular, separación de responsabilidades, persistencia relacional y validación de reglas de negocio desde el servidor. La siguiente etapa debe centrarse en mejorar la experiencia conversacional, la integración de voz y la calidad del prototipo para validarlo en un contexto real de restaurante.

16. Anexos sugeridos

- Repositorio GitHub: [AGREGAR ENLACE DEL REPOSITORIO PRIVADO O PÚBLICO, SI CORRESPONDE].
- Capturas de commits y git push.
- Capturas de Supabase con las tablas creadas.
- Capturas del frontend en ejecución.
- Capturas de pruebas por curl, Thunder Client o Postman.
- Captura del build exitoso del backend y frontend.
- Captura del prototipo en Stitch IA.

Checklist de entrega para el Avance 2

Elemento	Qué agregar	Estado
Código fuente	Repositorio GitHub con commits ordenados.	[PENDIENTE / COMPLETAR]
Base de datos	Capturas de tablas y datos cargados en Supabase.	[PENDIENTE / COMPLETAR]
Backend	Capturas de endpoints funcionando.	[PENDIENTE / COMPLETAR]
Frontend	Capturas de la interfaz con Leo y micrófono.	[PENDIENTE / COMPLETAR]
Prototipo Stitch	Capturas o enlace del prototipo visual.	[PENDIENTE / COMPLETAR]
PPT	Presentación resumida para exposición.	[PENDIENTE / COMPLETAR]
Demo	Flujo preparado: crear sesión → leer menú → pedir producto → repetir pedido → confirmar.	[PENDIENTE / COMPLETAR]